

RTOS PT.2

hi, csloot. i didnt notice that i have to do it from scratch. i already installed the headless, and install zsh, copy some of my configs there. but i will still try to do the assignment

1. done it, i buy a raspberry pi 2 for 15 euro

2. i use a raspberry pi

3. sudo adduser

basically the raspberry pi image flasher already asked to create a user from the start. but yeah i can make a user by utilizing `sudo adduser (blablala)` and i will set a password by logging in to the new user, and set `passwd` to set the user password.

not to mention but dont use `sudo` on `passwd` because then you will change the root user password

4. to be honest, i didnt find pi user. maybe its removed already ?

5. aha, this is interesting

i already installed raspberry pi lite, and i already turn it on. tho i run out of lan cable so i plug tplink archer T2U nano, and installed the driver. then i run it open 24/7 while i am out for vacation.

and basically i have a homeserver, and that home server is running tailscale.
my mac that i bring on my vacation also have tailscale, but not the rpi tho

so what i did is i ssh to my home server, then i ssh from my server to my rpi
so its a double SSH

1. sudo for super user do
2. apt-get means fetch from aptitude server (i prefer yay and brew btw). aptitude is like a package server
3. install to install the package, sometime you can also use apt-get to fetch package details before installing
4. iftop htop atop nmap netcat, its the package name
netcat which netcat tho ? traditional or openbsd? maybe i will do traditional done.

8. shutdown

shutting down usually with `sudo shutdown -h`, or you can also make your own power button and connect it with the gpio pin.

if you unplug it, maybe you are lucky one or two times, but your sd card would die fast if you did this. heck, you can even encounter os failing or your data is lost because there might be some data in your ram, and you unplug it, byebye data

9. uname ?

```
→ ~ uname -a
```

```
Linux 1770np-pi 6.12.75+rpt-rpi-v7 #1 SMP Raspbian 1:6.12.75-1+rpt1 (2026-03-11)
armv7l GNU/Linux
```

- linux is my os is linux based, if i were to run it on my mac it would say i am on darwin kernel
- 1770np-pi is my hostname. yes this is also configured by the image flasher raspberry pi
- 6.12.75+rpt-rpi-v7 is the kernel, oh yeah the kernel headers is already built by the flasher nowadays, so the raspberrypi kernel header installer wont work anymore.
- the #1 smp, i dont know to be honest, google didnt give me an answer as well
- raspbian 1:6.12.75-1+rpt1 is my distro flavour
- 2026-03-11 is the birth date of the os
- armv7l is the architecture of the raspberry pi
- GNU / Linux is ? idk the name maybe ?

10. masyallah. basic linux command

cat, cd, ls, df, echo, find, finger, free, grep, groups, ps, kill, man, mv, ping, pwd, top, lsblk

- cat to see the text inside a file
- cd is change directory
- ls id to list the dir (nice, ls -la goat)
- df for disk usage
- echo is to print smth, also used to see some enviroment variables
- find is to find a file
- finger, to see who logged in to the system
- free is to display ram usage
- grep is to filter an output
- groups to see the user groups
- ps to see what is running on the device
- kill of course to kill a service or an app, tho i use pkill usually
- man, goat, the manual
- mv to move file, give -r for repetitive, moving folder
- ping to see are you alive to the ip / dns name of the target amchie
- pwd to see the working directory
- top is task manager, tho i preffer bpytop, htop

- lsblk, to see what drive mounted to where, or what drive are connected to the device

11. why ? can i do it in person so i dont have to lsblk each of the directory ?

12. visudo ?

okay visudo is like the sudoers file, sudo means super user do and it used to abbreviate user privilege so the user can actually do something that is out of its league, for example destroying your machine if FBI come. if you type `rm -rf /` without sudo, it will say permission is denied, but if you type `sudo rm -rf /`.

13. logging ?

14. yes its correct

yes its correct, i use `date` commands, ofc

15. yeah, a bit of a coding

so basically i

1. installed neovim to edit file
2. make a new directory
3. touch something, like `touch main1.cpp`
4. cpp have 2 thing that you can take, 1 is argc, one more is argv. i take both but in this case.i only need argv[1]
5. make a normal cpp code to print hello argv[1]
6. this is the whole code

```
#include <stdio.h>

int main(int argc, char ** argv) {
    printf("hello, %s\n", argv[1]);
}
```

7. run `gcc main1.cpp` , you can also do -o for output place and name
8. run it, `./a.out (yourname)`

a.out
main1.cpp

```
1
2
3 #include <stdio.h>
4
5 int main(int argc, char ** argv) {
6     printf("hello, %s\n", argv[1]);
7 }
8 |
```

```
RPI-2B - ཨོ་མ་རྩི་པ་ལྷོ་ལྷོ་ལྷོ་ལྷོ་ OM MANI PADME HUM ཨྲ་མ་ལྷོ་ལྷོ་ལྷོ་
→ study gcc main1.cpp
RPI-2B - ཨོ་མ་རྩི་པ་ལྷོ་ལྷོ་ལྷོ་ལྷོ་ OM MANI PADME HUM ཨྲ་མ་ལྷོ་ལྷོ་ལྷོ་
→ study ./a.out name
hello, name
RPI-2B - ཨོ་མ་རྩི་པ་ལྷོ་ལྷོ་ལྷོ་ལྷོ་ OM MANI PADME HUM ཨྲ་མ་ལྷོ་ལྷོ་ལྷོ་
→ study
```

9.

17. benchmark

```
RPI-2B - ཨོ་མ་རྩི་པ་ལྷོ་ལྷོ་ལྷོ་ལྷོ་ OM MANI PADME HUM ཨྲ་མ་ལྷོ་ལྷོ་ལྷོ་
→ download time ./a.out
k=-520093696
./a.out 50.75s user 0.03s system 99% cpu 50.826 total
ROG - ཨོ་མ་རྩི་པ་ལྷོ་ལྷོ་ལྷོ་ལྷོ་ OM MANI PADME HUM ཨྲ་མ་ལྷོ་ལྷོ་ལྷོ་
→ download time ./benchcpu
k=3774873600
./benchcpu 1.83s user 0.00s system 99% cpu 1.840 total
```

he ? doesnt make sense.

18. endless void

okay endless is basically a c code that is like endless. like its name, so its actually a good tool to explain about how to put a things to suspend and bring it back and put it in background etc, and see

what is going on in the background

ps - what program is on now

top - bpytop, but less fancy, lets u see what is happening inside the pico now, like the process, pid, and any important stats like cpu usage

fg - if u ctrl + z in the a.out, it would actually suspend it, to make it come again in the terminal, type fg.

bg - same like fg, but it start in the background. then if this if you want to kill it you type ->

kill - to kill a process, based on their pid. you can also do pkill to do it using your name, rfkill to turn on and off radio like bluetooth, wifi, etc.

19. AddressSpace.c: Memory Layout and ASLR

After compiling and running AddressSpace.c multiple times, here is what i actually got:

- Address Ranges (Example Output):
 - Text (Code): 0x55855f633169 (main)
 - Initialized Data: 0x55855f636030 (variable a)
 - BSS (Uninitialized Data): 0x55855f636038 (variable b)
 - Heap: 0x55858363f010 to 0x558583681000 (managed by malloc and sbrk)
 - Stack: 0x7ffee0caf6f0 (managed by alloca and local variables)
- Differences:
 - (a) Data vs. BSS Segment: The Data segment stores global/static variables that are initialized in the source code (like `int a = 0xaaaa;`). The BSS segment stores global/static variables that are uninitialized or zero-initialized (like `int b;`).
 - (b) malloc vs. alloca: malloc allocates memory on the heap, which remains until freed. alloca allocates memory on the stack within the current function's frame, and it is automatically reclaimed when the function returns.
 - (c) Stack vs. Heap: The stack is used for automatic storage (local variables, function calls) and grows downwards. The heap is used for dynamic allocation and grows upwards towards the program break (brk).
- ASLR Observation: The output is not the same each time the program is executed. This is due to Address Space Layout Randomization (ASLR), a security feature that randomizes the memory locations of the stack, heap, and libraries to prevent predictable memory-based attacks.

20. ProcessLayout.c: Threads and Shared Memory

- Comparison (N=3 vs. N=2):

Changing N from 3 to 2 reduces the number of threads created. The output shows fewer tid and stack addresses, and the pmap output shows one less anonymous memory region.
- Memory Sharing:
 - Shared: All threads share the same text (code), data/BSS, and heap. The sbrk calls increase the common heap available to all.
 - Unique: Each thread has its own unique stack (as shown by different stack= addresses) and Thread ID (TID). This allows threads to have their own local variable context while sharing global data.

21. Fork.c: Process Cloning

- Explanation: The fork() system call creates a new process (child) that is an exact copy of the parent. The child executes the echo Foo command using execv.
- Modifications for top: Because the original program finishes almost instantly, I added sleep(30) to both the parent and child sections.
- Observation: Running ./Fork & followed by ps -u \$USER showed two active Fork processes. This confirms the parent and child are running concurrently as separate process entries in the system.

22. Pipe.c: Unnamed Pipes

- (i) Output Variance: The output varies because the parent seeds the random number generator with time(NULL). If runs are separated by at least one second, the seed changes, resulting in a different random number.
- (ii) Four close() calls: pipe() creates two file descriptors (read and write). fork() duplicates them, resulting in four total descriptors across the two processes.
 1. The parent closes the read end (it only writes).
 2. The child closes the write end (it only reads).
 3. The child closes its read end after finishing.
 4. The parent closes its write end after finishing.This ensures proper EOF signaling and resource management.

23.Fifo.c: Named Pipes

- Similarities/Differences: Both Pipe.c and Fifo.c use pipes for IPC. However, Pipe.c uses an unnamed pipe (invisible to other processes), while Fifo.c uses a named pipe (created with mkfifo) that appears in the file system.
- Is Command Recognition: You can recognize a named pipe in the ls -l output by the leading p in the permission string:

```
1 prw-r--r-- 1 user group 0 May 8 00:49 myfifo
```

24. Users, Groups, and Permissions

- Commands Explanation:
 - ls -la: Lists all files in the current directory, including hidden ones (starting with .), in a long format showing permissions, owners, size, and timestamps.
 - chown: Changes the owner and/or group of a file or directory. (e.g., sudo chown user:group file).
 - chmod: Changes the access rights (permissions) of a file. It uses octal numbers (e.g., 755) or symbolic modes (e.g., u+x).
- Access Rights Demonstration (Simulated):

Suppose we have users alice and bob in group lab.

1. -rw----- alice lab : Only alice can read/write. bob gets "Permission denied".
2. -rw-r----- alice lab : alice can read/write; bob (as part of lab) can only read.
3. -rwxr-xr-x alice lab : Everyone can read and execute; only alice can write.
4. ---w----- alice lab : Only alice can write (rarely used alone).
5. -rw-rw-rw- alice lab : Everyone can read and write.
6. -r----- alice lab : Read-only for alice; no one else has access.

25. In.sh: Hard vs. Symbolic Links

- Command: `sh -x In.sh >& In.log&` runs the script in the background with "execution trace" (-x), redirecting both standard output and error to `In.log`.
- What `In` does: It creates links between files. By default, it creates hard links; with `-s`, it creates symbolic (soft) links.
- Analysis of a, b, and c:
 - a and b (Hard Links): They share the same inode number (6048640). They are two names for the exact same data on the disk. Deleting a does not destroy b because the data remains as long as at least one hard link exists.
 - c (Symbolic Link): It has a different inode (6048642) and points to the name "a". When a and b were deleted, c became a broken link ("No such file or directory"). When a new a was created, c started working again because it points to the name, not the data.

26. Readdir.c: Outputting File Size

I modified the code to use the `stat()` function. For each entry found by `readdir()`, the program now calls `stat()` to retrieve the `st_size` field from the file's metadata.

Modified Source Code (Key Section):

```
1 struct stat st;
2 char fullpath[1024];
3 // ... inside while loop ...
4 snprintf(fullpath, sizeof(fullpath), "%s/%s", argv[1], dp->d_name);
5 if (stat(fullpath, &st) == 0) {
6 printf("%8d %c %10lld %s\n", (int)dp->d_ino, t, (long long)st.st_size,
dp->d_name);
7 }
```

27. USB Stick and Kernel Messages

- `sudo fdisk -l`: Lists all disk partitions. Before the USB, you see only the internal drive. After, a new device like `/dev/sdb1` appears.
- `dmesg`: Shows the kernel ring buffer. When you plug in a USB, you see messages about a "New high-speed USB device," the vendor/model name, and the device node assigned (e.g., `[sdb] Attached SCSI removable disk`).
- Location: USB drives are usually found under `/dev/` (as block devices) and are automatically mounted to `/media//` or `/mnt/` in modern Linux distributions.
- `lsusb`: Lists all USB controllers and connected devices (keyboard, mouse, hubs, USB sticks).
- Safe Unplugging: You must unmount the device (e.g., `umount /dev/sdb1` or use the "Eject" button in the GUI) to ensure all pending write operations are completed and the file system is not corrupted.

28. nmap and netcat

- Discovery: Running `ss -ltnu` (since `nmap` was not available) revealed that port 22 is listening.
- Banner Grabbing: Running `echo "" | nc -v localhost 22` returned: `SSH-2.0-OpenSSH_10.2`
- Findings: The open port 22 belongs to the OpenSSH service. The version string `SSH-2.0-OpenSSH_10.2` is the "welcome banner" that identifies the protocol version and the software being used.

29. Reflection/Conclusion

These assignments provide a hands-on transition from high-level programming to low-level system interactions. By exploring memory layouts (`AddressSpace.c`), process creation (`Fork.c`), and IPC (`Pipe.c/Fifo.c`), one gains a concrete understanding of how the Operating System manages resources. Tasks involving file systems (`ln.sh`) and hardware (`lsusb`) bridge the gap between abstract data and physical hardware management, which is essential for mastering Linux.